DLMDSEDE02 Project: Data Engineering Finalization Phase Report

Submitted By

Joyan Sharukh Bhathena

Matriculation: 9213297

Table of Contents

Kisan Call Centre Data Pipeline Finalization Report

This report presents the final implementation outcomes of the Kisan Call Centre (KCC) Farmer Queries Processing Pipeline, a microservices-based data engineering system designed to process approximately 9 million farmer query records from the Indian Government's Open Data Portal. The project successfully demonstrates scalable data ingestion, processing, and visualization capabilities while highlighting key technical challenges and their solutions.

Video walkthrough: https://www.awesomescreenshot.com/video/40567408?key=0026f2118d07937208ee2145f1464dbb

Link to github repository: https://github.com/Joyan9/kcc-farmer-queries-pipeline

1. Project Overview and Objectives

1.1 System Architecture

The implemented solution follows a **microservices architecture** comprising four main components:

- **Ingestion Service**: dlt-based (data load tool - python library for data ingestion) data extraction with API integration
- **Processing Service**: Spark-based ETL transformation into star schema
- **Visualization Service**: Jupyter Lab-based analytics dashboard
- **Testing Service**: Comprehensive test suite for pipeline validation

1.2 Technical Requirements Fulfillment

- **Microservices Architecture**: Successfully implemented four independent, containerized services
- **Large Dataset Processing**: Handles 9+ million records with configurable batch limit
- **Scalable Design**: Docker-based deployment with orchestration capabilities
- **Data Quality**: Data cleaning, PII masking, and validation processes

2. Implementation Phase

2.1 What Went Well

2.1.1 Successful Architectural Decisions

- **Microservices Approach**: Clear separation of concerns enabled independent development and testing

- **Docker Containerization**: Eliminated dependency conflicts and ensured consistent deployment
- **Star Schema Implementation**: Optimized analytical queries and dimensional modeling

2.1.2 Technical Achievements

- **Flexible Ingestion Modes**: Support for both incremental and backfill operations
- **Data Format Optimization**: Parquet format selection improved Spark compatibility
- **Performance Optimization**: Configurable row limits prevented resource exhaustion

2.2 Challenges Encountered and Solutions

2.2.1 Data Ingestion Challenges

Challenge 1: Data Format Compatibility

- **Problem**: Initial JSON format caused complex unnesting issues in Spark processing
- **Impact**: Data corruption and processing failures
- **Solution**: Migrated raw data storage to Parquet format for better compression and Spark compatibility
- **Lesson Learned**: Format selection should consider downstream processing requirements

Challenge 2: API Performance Limitations

- **Problem**: Backfill operations took excessive time due to API pagination (10 rows/call default)
- **Impact**: Pipeline execution times became impractical for large datasets
- **Solution**: Implemented configurable row limits (max 50,000 per month) and custom pagination logic
- **Lesson Learned**: Early performance testing prevents late-stage bottlenecks

2.2.2 Data Processing Challenges

Challenge 3: PII Data Handling

- **Problem**: Inconsistent patterns in personally identifiable information (names, phone numbers)
- **Impact**: Difficulty in implementing comprehensive data masking
- **Solution**: Developed regex-based PII masking with multiple pattern recognition
- **Lesson Learned**: Data governance requirements should be established early in the design phase

Challenge 4: Surrogate Key Generation

- **Problem**: Spark's distributed nature complicated surrogate ID creation for dimensions
- **Impact**: Performance degradation due to data shuffling to head node
- **Solution**: Optimized window functions and partition strategies

- **Lesson Learned**: Distributed computing constraints require specialized design patterns

2.2.3 Visualization Limitations

Challenge 5: Large Dataset Visualization

- **Problem**: Pandas memory limitations with million-record datasets
- **Impact**: Required subset sampling for visualization
- **Solution**: Implemented DuckDB for high-performance analytics queries
- **Alternative Considered**: Power BI was deemed too complex for project scope

3. System Quality Assessment

3.1 Reliability

- **Error Handling**: Exception handling in all microservices
- **Data Validation**: Multi-layer validation from ingestion through processing
- **Recovery Mechanisms**: Support for reprocessing and backfill operations
- **Testing Coverage**: Unit tests, integration tests, and end-to-end validation

3.2 Scalability

- **Horizontal Scaling**: Docker containers can be replicated across multiple nodes
- **Configurable Limits**: Adjustable batch sizes prevent resource exhaustion
- **Modular Design**: Independent services can be scaled based on demand
- **Performance Optimization**: Spark-based processing leverages distributed computing

3.3 Maintainability

- **Modular Architecture**: Clear separation of concerns facilitates updates
- **Documentation**: Clear README and architectural diagrams
- **Code Organization**: Consistent structure across all microservices
- **Configuration Management**: Externalized parameters for easy modification

4. Data Security and Governance Recommendations

4.1 Current Implementation

- Basic PII masking using regex patterns
- Local file system storage for processed data
- Container-level isolation for services

4.2 Recommended Enhancements

- **Enhanced PII Protection**: Implement advanced anonymization techniques

- **Cloud Storage Migration**: Move raw data to secure object stores (S3, Azure Blob)
- **Access Control**: Implement RBAC for database access
- **Network Security**: VPN or private network deployment
- **Audit Logging**: Comprehensive access and modification tracking
- **Data Encryption**: At-rest and in-transit encryption implementation

5. Skills Development and Learning Outcomes

5.1 Technical Skills Acquired

1. **Microservices Architecture**: Designed and implemented loosely coupled, independent services
2. **Container Orchestration**: Learnt Docker and Docker Compose for deployment
3. **Dimensional Modeling**: Applied star schema principles for analytical workloads

5.2 Soft Skills Developed

1. **Adaptability**: Successfully pivoted from initial design when technical constraints emerged
2. **Empathetic Development**: Wrote maintainable code considering future developers
3. **AI Tool Utilization**: Effectively leveraged ChatGPT and Claude for problem-solving

6. Future Enhancements and Workflow Improvements

6.1 Immediate Improvements

- **Orchestration Framework**: Implement Apache Airflow for metadata tracking and workflow monitoring
- **Performance Monitoring**: Add execution time tracking and failure analysis

6.2 Streaming Data Integration Strategy

To introduce real-time streaming capabilities, the following approach is recommended:

- **Stream Processing Engine**: Leverage Spark Structured Streaming for consistency with batch processing
- **Message Queue**: Implement Apache Kafka for reliable data streaming
- **Lambda Architecture**: Maintain both batch and streaming paths for different use cases

7. Conclusion

The Kisan Call Centre Farmer Queries Processing Pipeline successfully demonstrated a production-ready data engineering solution capable of handling large-scale agricultural data processing. Despite encountering technical challenges particularly around data format

compatibility, API performance limitations, and PII handling, the project achieved all primary objectives through adaptive problem-solving and robust architectural decisions.

The implementation provides a solid foundation for agricultural data analytics while highlighting important considerations for future data engineering projects, including the critical importance of early performance testing, format selection based on downstream requirements, and comprehensive data governance planning.

The modular, containerized architecture ensures the system can evolve to meet changing requirements, including the potential integration of real-time streaming capabilities for enhanced agricultural decision support systems.